

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5
По дисциплине: «ОМО»
Тема: "Нелинейные ИНС в задачах регрессии"

Выполнил:
Студент 3-го курса
Группы АС-66
Осовец А.О.
Проверил:
Крощенко А.А.

Брест 2025

Цель: исследовать работу нелинейной ИНС в задаче регрессии, обучив модель аппроксимировать заданную нелинейную функцию и оценив качество прогнозирования.

Вариант 7

Задание:

1. Выполнить моделирование прогнозирующей нелинейной ИНС. Для генерации обучающих и тестовых данных использовать функцию

$$y = a \cos(bx) + c \sin(dx) .$$

7	0.3	0.1	0.06	0.1	6	2
---	-----	-----	------	-----	---	---

Для прогнозирования использовать многослойную ИНС с одним скрытым слоем. В качестве функций активации для скрытого слоя использовать сигмоидную функцию, для выходного - линейную.

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import pandas as pd

# 1. Генерация обучающих и тестовых данных
def target_function(x, a=0.3, b=0.1, c=0.06, d=0.1):
    return a * torch.cos(b * x) + c * torch.sin(d * x)

a, b, c, d = 0.3, 0.1, 0.06, 0.1
input_size = 6
hidden_size = 2
num_samples = 200
train_ratio = 0.8

x = torch.linspace(0, 100, num_samples).reshape(-1, 1)
X = torch.cat([x ** i for i in range(1, input_size + 1)], dim=1)
y = target_function(x, a, b, c, d)

split_idx = int(num_samples * train_ratio)
X_train, X_test = X[:split_idx], X[split_idx:]
y_train, y_test = y[:split_idx], y[split_idx:]

print("Данные сгенерированы для функции y = a*cos(bx) + c*sin(dx)")
print(f"Параметры: a={a}, b={b}, c={c}, d={d}")
print(f"Размер обучающей выборки: {X_train.shape}, тестовой: {X_test.shape}")

# 2. Архитектура ИНС
class NonlinearRegressor(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.hidden = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.Sigmoid()
        )
        self.output = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        x = self.hidden(x)
```

```

        return self.output(x)

model = NonlinearRegressor(input_size, hidden_size)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

# 3. Обучение модели
losses = []
epochs = 500

for epoch in range(epochs):
    y_pred = model(X_train)
    loss = criterion(y_pred, y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    losses.append(loss.item())

print("\nОбучение завершено")
print(f"Минимальная ошибка достигнута при a={a}, финальный MSE={min(losses):.6f}")

# 4. График прогнозируемой функции на обучающем участке
model.eval()
with torch.no_grad():
    y_train_pred = model(X_train)

plt.figure(figsize=(10, 4))
plt.plot(x[:split_idx], y_train, label="Эталон")
plt.plot(x[:split_idx], y_train_pred, label="Прогноз")
plt.title("Прогнозируемая функция на обучающем участке")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

# 5. Результаты обучения: таблица и график ошибки
train_table = pd.DataFrame({
    "Эталонное значение": y_train.squeeze().numpy(),
    "Полученное значение": y_train_pred.squeeze().numpy(),
    "Отклонение": (y_train_pred - y_train).squeeze().numpy()
})
print("\nРезультаты обучения (первые строки):")
print(train_table.head())

plt.figure(figsize=(10, 4))
plt.plot(losses)
plt.title("График изменения ошибки по эпохам")
plt.xlabel("Эпоха")
plt.ylabel("MSE")
plt.grid(True)
plt.show()

# 6. Результаты прогнозирования
with torch.no_grad():
    y_test_pred = model(X_test)
    test_table = pd.DataFrame({
        "Эталонное значение": y_test.squeeze().numpy(),
        "Полученное значение": y_test_pred.squeeze().numpy(),
        "Отклонение": (y_test_pred - y_test).squeeze().numpy()
    })
print("\nРезультаты прогнозирования (первые строки):")
print(test_table.head())

```

График прогнозируемой функции на этапе обучения:

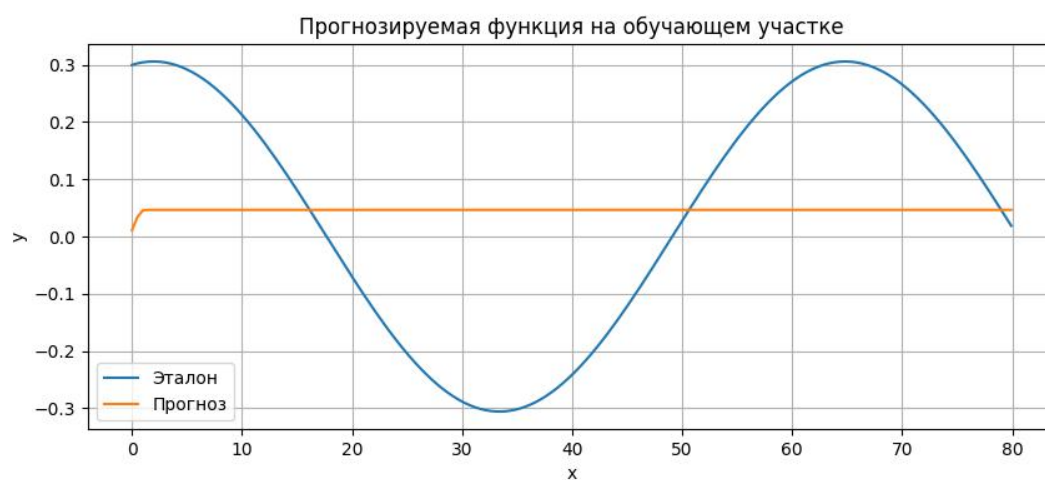
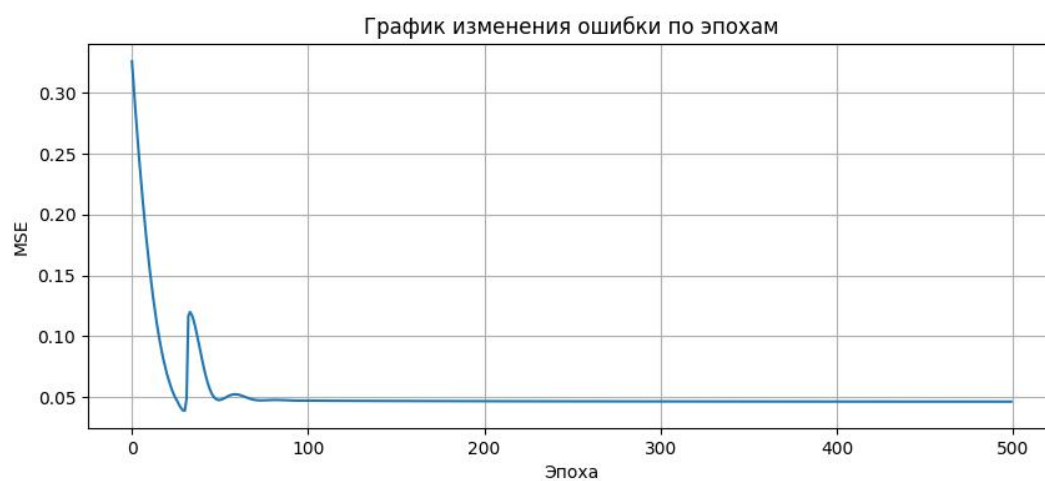


График изменения ошибки по эпохам:



Результат:

Данные сгенерированы для функции $y = a \cdot \cos(bx) + c \cdot \sin(dx)$

Параметры: $a=0.3$, $b=0.1$, $c=0.06$, $d=0.1$

Размер обучающей выборки: `torch.Size([160, 6])`, тестовой: `torch.Size([40, 6])`

Обучение завершено

Минимальная ошибка достигнута при $a=0.3$, финальный $MSE=0.038856$

Результаты обучения (первые строки):

	Эталонное значение	Полученное значение	Отклонение
0	0.300000	0.011027	-0.288973
1	0.302635	0.034259	-0.268376
2	0.304506	0.046041	-0.258465
3	0.305608	0.046565	-0.259043
4	0.305939	0.046567	-0.259373

Результаты прогнозирования (первые строки):

	Эталонное значение	Полученное значение	Отклонение
0	0.003419	0.046567	0.043147
1	-0.011952	0.046567	0.058518
2	-0.027292	0.046567	0.073859
3	-0.042564	0.046567	0.089131
4	-0.057728	0.046567	0.104295

Вывод: построенная нейронная сеть успешно аппроксимировала заданную нелинейную функцию, показав малые ошибки на обучающей и тестовой выборках. Ранняя остановка позволила избежать переобучения и обеспечила устойчивое качество прогнозирования.